YELDAR

Slide 1 / Kazakh Words. New Sound.

Hello, we're Yeldar, Aisha and Dimash.

Our project is a local Kazakh music-generation prototype. We wanted to take a description of music, add Kazakh lyrics, and generate an audio result.

Getting that working involved much more than writing a prompt. We had to find suitable recordings, check them, prepare training data, adapt an existing model, and make the process run on our own computers.

Today, we'll explain how we built it, show evidence from training, and play examples generated by the system.

NEXT: Aisha / Slide 2

AISHA

Slide 2 / First the Dombra. Then the Voice.

We started by looking for material that represented the sound we wanted, especially the dombra.

We found online videos through Wikimedia Commons and recordings in public datasets. We checked their sources and usage terms before deciding whether they were suitable.

For the archive expansion, we reviewed 116 recordings by ear. We separated solo dombra, dombra with vocals, and orchestral recordings.

We used 59 dombra-only recordings to create 111 new training clips. Vocals required additional work: the words had to match exactly what was sung in each clip. We held back 27 vocal recordings because their lyrics still needed verification.

NEXT: Aisha / Slide 3

AISHA

Slide 3 / Hugging Face. Aligned Kazakh Vocals.

For the later vocal stage, we used the Kazakh Songs ASR dataset from Hugging Face.

It provided isolated singing with aligned lyrics, which helped address the transcription problem. We accessed it through an approved account and prepared the data locally.

The source contained 3,013 segments. After filtering and removing exact duplicates, 2,977 remained.

We separated these into training, validation and test sets. Clips from the same identified song stayed together, helping reduce overlap between training and evaluation.

NEXT: Yeldar / Slide 4

YELDAR

Slide 4 / What We Started With. What We Trained.

Our foundation was ACE-Step, a pretrained music-generation system developed by ACE Studio and StepFun.

It already contains general music-generation capabilities. We adapted it to our Kazakh material using LoRA.

LoRA trains a smaller set of additional weights called an adapter. In our final experiment, approximately 5.5 million parameters were trainable, while the original model stayed frozen.

That distinction is important: we built the local preparation, training and generation workflow, and trained our adapter. We did not train the underlying foundation model from scratch.

Our baseline audio was also generated locally, using ACE-Step before applying our adapter.

NEXT: Dimash / Slide 5

DIMASH

Slide 5 / Many Runs. One Working Pipeline.

This project involved several experiments, rather than one training session.

We estimate over 300 minutes across training experiments. That is our retrospective estimate. The final run has a separate measured time of 66.3 minutes.

That final run completed 3,088 updates. Every one of the 2,316 eligible new training clips was used once, alongside repeated updates using nine original music clips.

We saved intermediate checkpoints so we could reload the model at different stages. Downloading, preparing data, troubleshooting and generating examples took additional time.

NEXT: Yeldar / Slide 6

YELDAR

Slide 6 / Recorded Steps. Saved Model Weights.

Here is evidence from the actual training run.

The graph comes from recorded training-loss values. Each point averages a group of 64 updates, making the overall pattern easier to see.

We also saved four checkpoints. Their checksums are different, and we verified those checksums against the actual files.

The links open the training log, configuration and source code. These let us show what ran and what was saved, rather than relying only on screenshots or a finished audio clip.

The decreasing loss shows progress on the training objective. It does not, by itself, prove that the music sounds better.

NEXT: Dimash / Slide 7

DIMASH

Slide 7 / The Hard Parts.

The hardest work was making the data and software usable together.

We checked recordings for decoding problems, silence, clipping and duplicates. For vocals, automatic lyric drafts were not reliable enough to accept without checking.

We also needed to preserve Kazakh characters correctly throughout processing.

Hardware was another constraint. For the final laptop run, we used four-bit model weights, a small adapter and gradient checkpointing to reduce memory requirements.

The training timer does not include all that preparation and troubleshooting.

NEXT: Aisha / Slide 8

AISHA

Slide 8 / From Our Prompt to Generated Music.

These are prerecorded examples generated locally with the same prompt, lyrics, seed and duration.

First, we'll play the baseline from the pretrained model.

[Play approximately 20 seconds of the baseline, then pause.]

Now we'll play the output with our trained adapter applied at 25 percent strength.

[Play the same 20-second interval of the adapted output, then pause.]

That percentage controls the adapter's influence during generation. Both examples demonstrate the working pipeline; we are not presenting them as proof of consistently improved quality.

NEXT: Yeldar / Slide 9

YELDAR

Slide 9 / Describe the Music. Add Kazakh Words.

Choosing the right input format was another important decision.

We investigated VocalRender, a different singing system that requires words, musical pitches and note durations. Preparing those annotations for our recordings would have required detailed work at the note and syllable level.

ACE-Step gave us a more practical route: a descriptive prompt plus lyrics.

The prompt shown here is the actual description used for these examples. We specified the style, voice and accompaniment, then supplied Kazakh lyrics. We left BPM and key unspecified and disabled automatic prompt rewriting.

This made our implementation a prompt-to-music workflow, rather than a system where we manually compose every note.

NEXT: Yeldar / Slide 10

YELDAR

Slide 10 / Our Local Build.

Our result is an ACE-Step-based Kazakh music prototype with a working local workflow.

We found and checked source material, prepared instrumental and vocal data, trained an adapter, saved the resulting weights, and generated audio examples.

We used open-source software, pretrained weights and existing datasets, and brought those components together for our project.

There is still work to do, particularly on pronunciation, musical consistency and broader evaluation. But we now have a functioning system and recorded evidence of how it was built.

Thank you. We're ready for your questions.